Report – BST with Go

Gerald Fattah

CS378H – Assignment 3

October 22nd, 2025

Overview:

The goal of this project was to detect which binary search trees (BSTs) constructed from different input lines are equivalent — meaning they contain the same values and would produce the same BST structure when inserted in the same order. The program supports a sequential version and several parallel versions that use goroutines, channels, locks, and worker pools to speed up hashing and comparison on larger inputs. To ensure correctness under concurrency, the Union-Find data structure was updated with mutex locks to prevent data races during parallel comparisons and merges.

Sequential Algorithm:

The baseline implementation reads each line of input, inserts the integers into a BST, and then performs these steps:

1. In-order traversal – Produces a sorted slice of values for each tree.
2. Hashing – The traversal output is turned into a SHA-1 hash.
3. Grouping by hash – Trees with different hashes are guaranteed to be different; only matching hashes are potential duplicates.
4. Deep comparison – For trees that share a hash, a pairwise comparison confirms whether the trees are truly equivalent.
5. Union-Find – Equivalence classes are formed so the final output can list groups of identical trees.

This version runs entirely in one thread and serves as the correctness reference before adding concurrency.

Hash Grouping:

After hashing, results must be grouped in a shared map keyed by the hash, which introduces synchronization. I implemented two approaches to handle this:

- Channel-collector design: Workers send (hash, id) pairs to a single goroutine that updates the map. This avoids locks entirely and guarantees correctness through serialization, but it can become a bottleneck because one goroutine must process all updates.

- Mutex-protected map: Multiple workers update the shared map concurrently while holding a mutex. This removes the single choke point and scales better with many workers, though contention can appear under heavy load.

On small inputs, both perform similarly, but on larger ones the difference is clear — the channel version is simpler and efficient until the collector saturates, while the mutex version spreads the work out more evenly at the cost of synchronization overhead.

Parallel Hashing

Hashing is an embarrassingly parallel task since each tree can be traversed independently. Two hashing modes were implemented:

- Goroutine-per-tree: Spawns one goroutine per tree. Great for coarse workloads but adds significant overhead on fine workloads due to excessive goroutine creation.
- Worker pool: Uses a fixed number of goroutines that receive tasks through a channel. This avoids overhead and produces smoother timing, especially for fine workloads.

Parallel Comparison:
After hash grouping, only trees within the same hash bucket need to be compared more deeply. Two comparison strategies were implemented: a goroutine mode, which spawns one goroutine per comparison, and a worker pool mode, which uses a bounded channel and a fixed number of workers to handle all comparisons.

The Union-Find structure was updated to include a mutex, ensuring that the Find, Union, and Groups operations are thread-safe. Without locking, concurrent union operations caused data races and led to inconsistent parent arrays. Introducing fine-grained locking resolved these issues and produced consistent, race-free results when tested with go run -race.

On simple.txt, the dataset is so small that synchronization and goroutine overhead dominate the runtime, leaving little room for parallel speedup. On fine.txt, the worker-pool version performs more consistently since it reuses goroutines and avoids scheduling spikes. Both comparison modes safely update the shared Union-Find structure under concurrent execution.

Results and Observations

The performance of the goroutine and worker pool implementations was evaluated using three datasets: simple.txt, coarse.txt, and fine.txt.

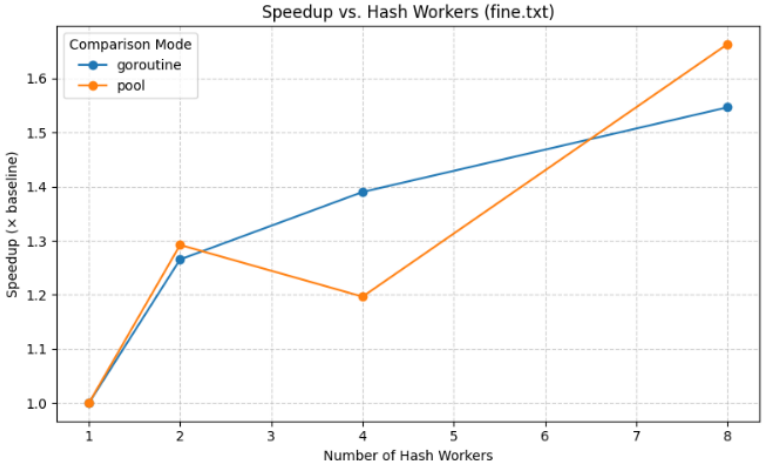
For simple.txt, the tree sizes were very small, and the work per task was minimal. Because of this, the overhead of managing concurrency outweighed any parallel benefit. Execution times for both goroutine and pool implementations were nearly identical to the sequential baseline. The graphs also showed irregular scaling, confirming that synchronization and task management costs dominated the runtime.

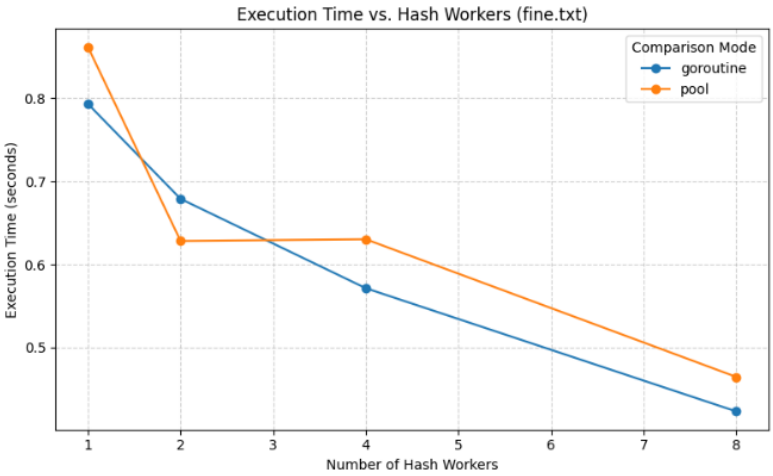
For coarse.txt, the larger trees made hashing and comparison operations expensive enough for concurrency to be worthwhile. Execution time dropped sharply as the number of hash workers increased, with clear speedup up to about four workers before leveling off. The goroutine implementation consistently ran slightly faster than the worker pool, showing lower scheduling overhead. Beyond four workers, both versions showed diminishing returns as the workload became fully parallelized.

For fine.txt, the dataset contained many small trees, resulting in mixed behavior. Both implementations saw gradual decreases in execution time as more workers were added, but the worker-pool version performed more efficiently overall, avoiding excessive goroutine creation. The two synchronization modes (channel vs. lock) behaved similarly at higher worker counts, where synchronization overhead dominated.

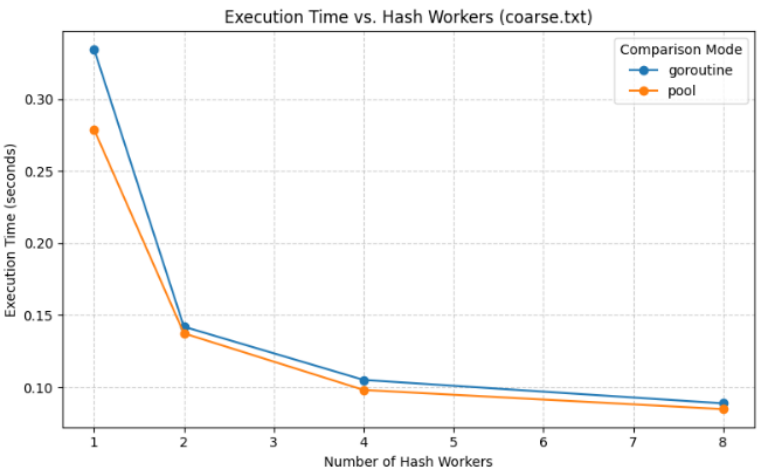
In summary, adding workers improved performance for larger datasets, particularly for coarse.txt, while simple.txt remained limited by concurrency overhead. The goroutine implementation generally achieved faster results, though the worker pool offered more predictable scaling for workloads with many small tasks.

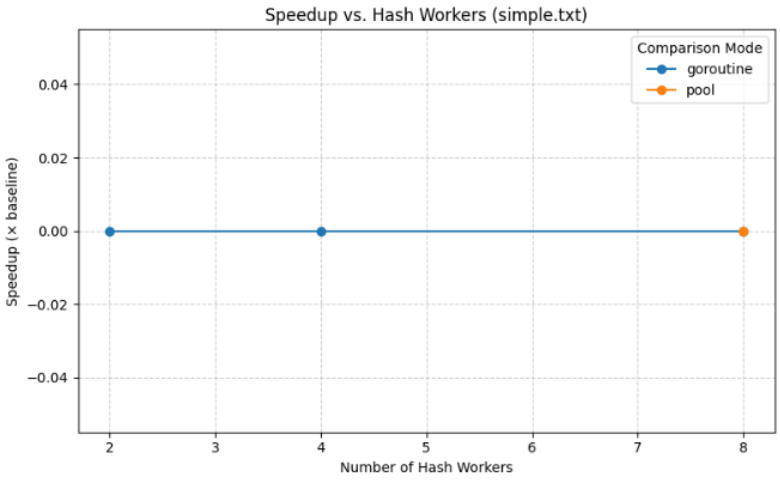
```
==================== RESULTS ====================

Input file: input/simple.txt
-----------------------------------------------------------
CompMode     HashWorkers  Total Time (s)
-----------------------------------------------------------
goroutine    1            0.000460
goroutine    2            0.000577
goroutine    4            0.000671
goroutine    8            0.000716
pool         1            0.000371
pool         2            0.000652
pool         4            0.000670
pool         8            0.000473
-----------------------------------------------------------

Input file: input/coarse.txt
-----------------------------------------------------------
CompMode     HashWorkers  Total Time (s)
-----------------------------------------------------------
goroutine    1            0.412133
goroutine    2            0.319186
goroutine    4            0.239770
goroutine    8            0.177094
pool         1            0.429271
pool         2            0.304827
pool         4            0.210682
pool         8            0.214895
-----------------------------------------------------------

Input file: input/fine.txt
-----------------------------------------------------------
CompMode     HashWorkers  Total Time (s)
-----------------------------------------------------------
goroutine    1            1.272373
goroutine    2            1.070496
goroutine    4            0.957813
goroutine    8            0.651615
pool         1            1.258791
pool         2            1.107427
pool         4            0.855266
pool         8            0.671919
-----------------------------------------------------------
===========================================================
```

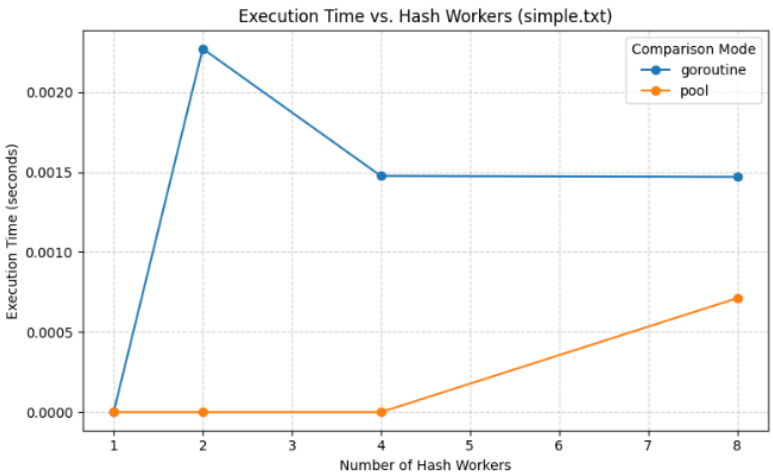

Speedup vs. Hash Workers (fine.txt)
Comparison Mode
goroutine
pool
Speedup (× baseline)
Number of Hash Workers


Execution Time vs. Hash Workers (fine.txt)
Comparison Mode
goroutine
pool
Execution Time (seconds)
Number of Hash Workers


Speedup vs. Hash Workers (coarse.txt)
Comparison Mode
goroutine
pool
Speedup (× baseline)
Number of Hash Workers


Execution Time vs. Hash Workers (coarse.txt)
Comparison Mode
goroutine
pool
Execution Time (seconds)
Number of Hash Workers


Speedup vs. Hash Workers (simple.txt)
Comparison Mode
goroutine
pool
Speedup (× baseline)
Number of Hash Workers


Execution Time vs. Hash Workers (simple.txt)
Comparison Mode
goroutine
pool
Execution Time (seconds)
Number of Hash Workers

Takeaways:

Across all three stages—hashing, hash grouping, and tree comparison—the sequential version served as the baseline for performance comparisons. Parallelizing the hash computation gave nearly linear speedups at low thread counts, especially on *coarse.txt*, where each tree is large and hashing dominates the runtime. In contrast, *fine.txt* saw smaller gains since the work per tree was lighter and the cost of managing goroutines and synchronization started to outweigh the hashing time.

During hash grouping, I initially expected the mutex-based version to scale better than the channel collector. However, the data showed that the channel version performed slightly better at low to moderate parallelism, likely because it avoided lock contention. As more workers were added, its single-threaded collector became a bottleneck, and the mutex approach caught up. Both methods still showed clear improvements over the sequential baseline, though neither achieved linear scaling.

In the comparison stage, the results matched expectations. The worker-pool approach was generally more stable and efficient on larger datasets like *coarse.txt* and *fine.txt*, where it reused goroutines and reduced scheduling overhead. The goroutine-per-comparison mode performed well at smaller scales but became less consistent as the number of workers increased. On *simple.txt*, performance differences were negligible across all configurations since the dataset was too small for concurrency to help.

Overall, parallelism delivered meaningful gains for larger workloads, with the most consistent improvements coming from the hashing and comparison stages. The worker-pool approach proved to be a practical balance between concurrency and overhead, while the goroutine-based method offered better peak performance when the workload was heavy enough to hide scheduling costs.

Time Spent: 22 hrs